IRBench method sweep

Benchmark version 3b92c1a, 10 runs, 500 questions

210 configurations were scored on all 10 runs through the benchmark's own `score.py`, and about 60 weaker ones only on the first 7 runs (marked below). The number compared everywhere is `above_floor`: the score minus the run's floor, where the floor is the best published list that ignores the question. Harness checks: the shipped `example_bm25` reproduces the README (0.258, +0.160 on `on_call`), a list built from the key scores 0.998 to 1.000 on every run, an empty list scores 0 and an audit hook stops any method that opens a key file.

Verdict: the benchmark is not exploitable

No strategy that ignores the question comes near real retrieval. The best question blind attack, a list learned from the other runs' keys, reaches +0.046, against +0.508 for the best honest method and +0.274 for plain BM25. List shaping does not inflate scores either: listing every id twice, adding every copy of the same text, flooding with tiny pieces near the 64 token minimum or mixing an event with its own blocks all score at or below the BM25 list they were built from.

attempt	what it does	above_floor
`x_writes_only_bm25`	BM25 keeping only blocks of file writes	0.342
`x_repeat_ids_bm25`	BM25 with every id listed twice	0.274
`x_all_copies_bm25`	every other copy of the same text after each hit	0.274
`ref_key_optimised_blind`	reference, not an attack: one list per run from its own key	0.270
`x_block_then_event_bm25`	each block followed by its whole event	0.207
`x_staged_big_event_first`	one big event per budget size placed first	0.198
`x_tiny_only_bm25`	only pieces of at most 64 tokens	0.148
`x_learned_blind_loro`	blind list from a model trained on the other runs' keys	0.046
`x_consensus_all`	blocks the run's questions retrieve most often	−0.002
`x_qtype_template`	fixed query per question type	−0.019
`x_asker_newest`	the asker's own steps, newest first	−0.043
`blind_top_agent`	best plain blind baseline	−0.039
`s_base_bm25stemstop`	honest reference	0.274
`c_rrf_kind_field_neigh_moves_dedup`	best honest method	0.508

Two results need a sentence each. The writes only filter beats plain BM25 because file writes hold most of the answers (62% of answer parts have full credit in a file write, though writes are 13% of blocks). It still retrieves for the question and stays well below the honest methods that use the same fact as a soft prior. The key optimised blind list shows the one weakness found: a single fixed list per run, ranked from the key by credit per token, scores as well as plain BM25, because credit sits in a few hub pieces such as status files and notebooks. So the published floor is not the best possible blind list. It does not make the benchmark gameable, because building that list needs the run's own key, and a list learned from the other nine runs' keys gets only +0.046.

Leaderboard (all 10 runs, mean above_floor)

rank	method	family	above_floor	score
1	`c_rrf_kind_field_neigh_moves_dedup`	combo	0.508	0.590
2	`c_rrf_kind_moves_dedup`	combo	0.480	0.563
3	`c_bm25_kind_field_neigh_moves_path_dedup`	combo	0.469	0.551
4	`c_bm25_kind_field_neigh_moves_dedup`	combo	0.468	0.550
5	`c_bm25_kind_field_neigh_moves50_dedup`	combo	0.467	0.549
6	`c_rrf_kind_field_dedup`	combo	0.457	0.540
7	`c_bm25_kind_field_neigh_moves_dedup_pertoken`	combo	0.456	0.539
8	`c_rrf_kind_neigh_dedup`	combo	0.456	0.538
9	`c_bm25_kind_writes_field_neigh_moves_dedup`	combo	0.450	0.533
10	`c_bm25_kind_field0.5_neigh_moves_dedup`	combo	0.449	0.532
11	`c_rrf_kind_dedup`	combo	0.446	0.528
13	`c_bm25_kind_moves_dedup`	combo	0.437	0.520
21	`s_kind_calls2`	structure	0.383	0.466
23	`s_kind_results0.3`	structure	0.382	0.464
28	`p_bm25_dedup_toevent`	packing	0.371	0.454
31	`s_dedup`	structure	0.370	0.453
44	`h_rrf60_bm25_bge_base_e5_small`	hybrid	0.330	0.413
45	`h_ce_msm6_top100_rrf_mix0.5`	hybrid	0.328	0.410
62	`d_bge_base_blocks`	dense	0.294	0.376
75	`bm25_stemstop_blocks`	lexical	0.274	0.356
121	`example_bm25`	lexical	0.225	0.308
7 runs	`e5_small`, `bge_small`, `gte_small`, `minilm`	dense	0.23 to 0.29	

Names read as a recipe. The first stage is `bm25` (BM25 with stemming and stopwords over blocks) or `rrf` (reciprocal rank fusion of that BM25 and the `bge_base` embedding model). Then `kind` multiplies scores by the step type (tool calls x2, tool results x0.3, prompts 0), `field` adds BM25 over a small field of agent name, step type and file touched, `neigh` adds a share of the best score in the same event and in the agent's previous and next step, `moves` follows file move commands for location questions so the answer points where a note sits now, and `dedup` drops repeated text.

What the top 10 do and why they score:

1. `c_rrf_kind_field_neigh_moves_dedup` fuses BM25 with dense retrieval and stacks all four structure steps. The dense half finds paraphrased answers BM25 misses, and it wins 9 of the 10 runs.
2. `c_rrf_kind_moves_dedup` is the same fused start with only the kind prior and move following, which carry most of the gain.
3. `c_bm25_kind_field_neigh_moves_path_dedup` is the best BM25 only stack; the path boost adds little over rank 4.
4. `c_bm25_kind_field_neigh_moves_dedup` is the full stack on BM25 and the reference point for that family.
5. `c_bm25_kind_field_neigh_moves50_dedup` follows moves from the top 50 instead of 20, with no real change.
6. `c_rrf_kind_field_dedup` drops neighbours and moves. It loses 0.13 on location questions but matches the winner on findings and decisions.
7. `c_bm25_kind_field_neigh_moves_dedup_pertoken` orders by score per token, which does not help once duplicates are gone.
8. `c_rrf_kind_neigh_dedup` shows neighbour expansion recovers the tool result next to a matching tool call.
9. `c_bm25_kind_writes_field_neigh_moves_dedup` adds a file write boost on top of the kind prior, which double counts and costs 0.02.
10. `c_bm25_kind_field0.5_neigh_moves_dedup` gives the field score more weight, which also costs 0.02.

What the results say about the benchmark

It separates methods. A bootstrap over questions puts about 0.025 either side of each mean, the winner beats every other top 30 method in 95% of resamples, and rankings agree across runs (Kendall tau 0.77 over 194 methods, Spearman 0.92). Inside the top 20 agreement is lower (tau 0.42), so methods within about 0.02 of each other should be read as ties.

Knowing the log's structure matters more than the text matcher. Swapping BM25 for dense or fused retrieval adds 0.02 to 0.06, while the step type prior, dedup and move following add 0.20 on top of BM25. Dense models alone (best `bge_base` 0.294) barely beat tuned BM25 (0.274), and a cross encoder reranker over the fused top

100 did not beat the fused list itself (0.328 against 0.330). Budget aware ordering adds nothing once duplicates are removed.

The hardest question types for the top method are state questions (0.338, 33 questions) and who and history questions (0.46). Location questions are the largest group (170) and the ones move following fixes. Only 1 question gets no credit from the best method. Of the 14 questions no honest method answered on the 7 run version, most were location questions whose answer is a later `mv` command. `on_call` a05 is debatable: it asks where a note was first written, and credit goes to a later move.

Not run on all 10 runs to save compute: the other five dense models, SPLADE (out of GPU memory at pilot), the larger cross encoders and cross encoder first stages for the stacks. Their 7 run results, where they exist, are in `results.csv` under `bench_version` eb53bdb.